Reviewer Pack — Tree-Depth Exponentiation Bounds Tseitin Resolution Length

Entry c965301cf284 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 15:18:18 UTC

Tree-Depth Exponentiation Bounds Tseitin Resolution Length

Entry ID: `c965301cf284` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-11 15:18:18 UTC

1. Conjecture

Field A (mathematical branch): Tree-Depth of Graphs **Field B** (complexity object): Resolution Proof Length for Tseitin Formulas

Statement:

> For any connected graph G with n vertices, the minimal Resolution proof length for the Tseitin formula over G is ≥ 2^{Ω(tree_depth(G))}.

Rationale (proposer's reasoning):

> Tree-depth captures the hierarchical complexity of graphs, which directly correlates with the number of resolution steps needed to derive contradictions in Tseitin formulas. Higher tree-depth implies more nested structures requiring exponential proof length.

Taxonomy category: `TSEITIN_RES` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `d48b101ebab2e83a`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from collections import defaultdict

def generate_random_graph(n):
    if n <= 1:
        return []
    edges = set()
    while len(edges) < n - 1:
        u, v = random.sample(range(n), 2)
        if (u, v) not in edges and (v, u) not in edges:
            edges.add((u, v))
    return list(edges)

def tree_depth(graph):
    if not graph:
        return 0
    adjacency_list = defaultdict(list)
    for u, v in graph:
        adjacency_list[u].append(v)
        adjacency_list[v].append(u)

    def dfs(node, parent):
        max_depth = 0
        for neighbor in adjacency_list[node]:
            if neighbor != parent:
                depth = dfs(neighbor, node)
                max_depth = max(max_depth, depth)
        return max_depth + 1

    return dfs(0, -1)

def tseitin_formula(graph):
    n = len(graph)
    literals = {f'x{i}': i for i in range(n)}
    clauses = []

    def add_clause(clause):
        if clause:
            clauses.append(clause)

    # Base case: each vertex is connected to at least one other vertex
    for u, v in graph:
        add_clause([-literals[u], literals[v]])
        add_clause([-literals[v], literals[u]])
```

```
    # Ensure each vertex has exactly two neighbors (simple cycle)
    for i in range(n):
        neighbors = [j for j in range(n) if (i, j) in graph or (j, i) in graph]
        if len(neighbors) != 2:
            return None

    # Tseitin encoding
    for u, v in graph:
        literal_uv = n + u * n + v
        add_clause([-literals[u], -literals[v], literal_uv])
        add_clause([literals[u], literal_uv])
        add_clause([literals[v], literal_uv])
        add_clause([-literal_uv, -literals[u]])
        add_clause([-literal_uv, -literals[v]])

    return clauses

def dpll(clauses):
    assignment = {}
    stack = []

    def is_satisfiable():
        for clause in clauses:
            if not any(lit in assignment and (assignment[lit] == 1) or (-lit in assignment and (as
                return False
        return True

    def backtrack():
        if not stack:
            return is_satisfiable()

        var, value = stack.pop()
        assignment[var] = value

        if is_satisfiable():
            return True

        del assignment[var]
        stack.append((var, -value))

        if is_satisfiable():
            return True

        stack.pop()
        return False

    for clause in clauses:
        unassigned_vars = [lit for lit in clause if lit not in assignment and -lit not in assignme
        if unassigned_vars:
            var = random.choice(unassigned_vars)
            stack.append((var, 1))

    return backtrack()

def run_trial(seed: int) -> dict:
```

```
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_length = 0
    instances_tested = 0

    for n in n_values:
        for _ in range(5):  # Ensure at least 5 instances per seed
            graph = generate_random_graph(n)
            if not graph:
                continue

            depth = tree_depth(graph)
            formula = tseitin_formula(graph)

            if formula is None:
                return {
                    "metric_name": "Resolution proof length",
                    "metric_value": float('inf'),
                    "instances_tested": instances_tested,
                    "conjecture_holds": False,
                    "counterexample": "mapping_undefined"
                }

            length = sum(1 for _ in range(100) if dpll(formula))
            total_length += length
            instances_tested += 1

    mean_length = total_length / instances_tested
    conjecture_holds = all(length >= 2 ** (0.5 * depth) for depth, length in zip(tree_depths, leng

    return {
        "metric_name": "Resolution proof length",
        "metric_value": mean_length,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 17 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_length = sum(res["metric_value"] for res in results) / len(results)
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_length:.2f} std=0.00 support_fraction=1.00")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_length:.2f} std=0.00 support_fraction={support_fracti
    else:
```

```
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d9a239b7.py", line 158, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d9a239b7.py", line 125, in run_trial
    depth = tree_depth(graph)
            ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d9a239b7.py", line 43, in tree_depth
    return dfs(0, -1)
           ^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d9a239b7.py", line 39, in dfs
    depth = dfs(neighbor, node)
            ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d9a239b7.py", line 39, in dfs
    depth = dfs(neighbor, node)
            ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d9a239b7.py", line 39, in dfs
    depth = dfs(neighbor, node)
            ^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 994 more times]
RecursionError: maximum recursion depth exceeded
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed due to recursion depth limits before collecting data; cannot assess conjecture validity. | next: Implement iterative DFS in tree_depth function to avoid recursion limits and re-run experiments

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111608
2	propose	ollama_remote	qwen3:8b	0	0	101263
3	propose	ollama_remote	qwen3:8b	0	0	98326
4	propose	ollama_remote	qwen3:8b	0	0	74168
5	propose	ollama_remote	qwen3:8b	0	0	89157
6	preregistration	ollama_remote	qwen3:8b	0	0	25110
7	novelty	ollama_remote	qwen3:8b	0	0	20735
8	novelty	ollama_remote	qwen3:8b	0	0	18960
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14542
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14353
11	judge	ollama_remote	qwen3:8b	0	0	21142

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 589363 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/c965301cf284.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c965301cf284.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c965301cf284.tar.gz` (if generated)